

Publish TypeScript Logger to GitHub

<https://gemini.google.com/app/f04aa91ee5b67be2>

User prompt: how to published a logger mofule with github typescript

Response: Here is a complete, step-by-step guide to writing a TypeScript logger module, hosting it on GitHub, and publishing it—either to the public **npm registry** or to **GitHub Packages**.

1. Initialize the Project

Create a directory for your module, open a terminal, and initialize Node.js and TypeScript:

```
mkdir my-logger
cd my-logger
npm init -y
npm install --save-dev typescript @types/node
npx tsc --init
```

2. Configure tsconfig.json

Set up TypeScript to output Javascript files (`dist/`) along with automatically generated `.d.ts` type definition files:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "declaration": true,
    "declarationMap": true,
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*"]
}
```

3. Write the Logger Code

Create a folder `src` and add `src/index.ts`:

```
export type LogLevel = 'info' | 'warn' | 'error' | 'debug';

export interface LoggerOptions {
  prefix?: string;
  enableTimestamp?: boolean;
}

export class Logger {
```

```

private prefix: string;
private enableTimestamp: boolean;

constructor(options: LoggerOptions = {}) {
  this.prefix = options.prefix || 'LOG';
  this.enableTimestamp = options.enableTimestamp ?? true;
}

private format(level: LogLevel, message: string): string {
  const timestamp = this.enableTimestamp ? `[${new Date().toISOString()}]` : '';
  return `${timestamp} [${this.prefix}] [${level.toUpperCase()}]: ${message}`;
}

info(message: string): void {
  console.log(this.format('info', message));
}

warn(message: string): void {
  console.warn(this.format('warn', message));
}

error(message: string): void {
  console.error(this.format('error', message));
}

debug(message: string): void {
  console.debug(this.format('debug', message));
}
}

```

4. Configure package.json

Update your package.json so consumer projects know where to entry point JavaScript files and type definitions.

Note: If publishing to **GitHub Packages**, your package name **must** start with @your-github-username/ (e.g., @octocat/my-logger).

```

{
  "name": "@your-username/my-logger",
  "version": "1.0.0",
  "description": "A simple TypeScript logger module",
  "main": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "files": [
    "dist"
  ],
  "scripts": {
    "build": "tsc",
    "prepare": "npm run build"
  },
  "repository": {
    "type": "git",

```

```
{
  "url": "git+https://github.com/your-username/my-logger.git"
},
"keywords": ["logger", "typescript", "logging"],
"author": "Your Name",
"license": "MIT"
}
```

Add a `.gitignore` and `.npmignore` to avoid committing build artifacts or uploading source files unnecessarily:

```
# .gitignore
node_modules/
dist/
```

5. Test the Build Locally

Before publishing, verify that your compilation runs smoothly:

```
npm run build
```

This generates the compiled JavaScript and `.d.ts` definitions in the `dist/` directory.

6. Push Code to GitHub

Initialize your Git repository and push your source code:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/your-username/my-logger.git
git push -u origin main
```

7. Choose How to Publish

Option A: Publish to standard npm registry

1. Log into your npm account in the terminal:

```
npm login
```

2. Publish the package (use `--access public` if using a scoped name like `@username/my-logger`):

```
npm publish --access public
```

Option B: Publish to GitHub Packages (GPR)

If you want to host the package natively inside your GitHub repository:

1. Add `publishConfig` to your `package.json`:

```
"publishConfig": {
  "registry": "https://npm.pkg.github.com"
}
```

2. Create a `.npmrc` file in your root folder:

```
@your-username:registry=https://npm.pkg.github.com
```

3. Generate a Personal Access Token (PAT) on GitHub with `write:packages` permission.

4. Log into GitHub Packages via CLI:

```
npm login --registry=https://npm.pkg.github.com
```

Username: your GitHub username

Password: your generated Personal Access Token (PAT)

5. Run:

```
npm publish
```

Optional: Automate Publishing with GitHub Actions

To automatically publish whenever you release or tag a new version, add `.github/workflows/publish.yml`:

```
name: Publish Package
```

```
on:
  release:
    types: [created]
```

```
jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          registry-url: 'https://registry.npmjs.org' # Or 'https://npm.pkg.github.com'

      - run: npm ci
      - run: npm run build
      - run: npm publish --access public
    env:
      NODE_AUTH_TOKEN: ${ secrets.NPM_TOKEN }
```

(For npm, save your NPM authentication token in GitHub repository secrets as `NPM_TOKEN`).

How Consumers Use Your Module

Once published, consumers can install and import your logger:

```
npm install @your-username/my-logger

import { Logger } from '@your-username/my-logger';

const logger = new Logger({ prefix: 'App' });
logger.info('Application started successfully!');
```

Sources

1. [Publishing npm Packages to GitHub Packages With Yarn - Ken Muse](#)
 2. [Step-by-Step: Building and Publishing NPM Packages — TypeScript | by Kavindu Madushanka | Medium](#)
 3. [Best Practices for Creating a Modern npm Package with Security in Mind | Snyk](#)
-